New Horizons®
Centro de Capacitación

Curso

Desarrollo Full Stack con C# y Angular

Docente: Edwin Barrientos



New Horizons®
Centro de Capacitación

Sesión 13 — Módulo 4:

Seguridad y Autenticación Backend con JWT



Objetivo del módulo: implementar autenticación segura basada en JSON Web Tokens (JWT) en ASP.NET Core Web API, dominando claims, password hashing con algoritmos robustos y protección de endpoints con el atributo [Authorize].



New Horizons®
Centro de Capacitación

Agenda

- Autenticación moderna en APIs: Tokens JWT vs Sesiones de servidor
- Anatomía de un JWT: Header, Payload (Claims estándar y personalizados) y Firma
- Password Hashing criptográfico seguro con PBKDF2 y Salt dinámico
- Configuración del pipeline en .NET 8: AddAuthentication y AddJwtBearer
- Endpoint POST /api/auth/login y protección de controladores con [Authorize]



New Horizons®
Centro de Capacitación

Sesión 1:

Seguridad y Autenticación Backend con JWT en ASP.NET Core

Visión general de la sesión



Hoy montamos la seguridad de un festival. Una taquilla que verifica tu entrada (login), una pulsera imposible de falsificar (JWT) y guardias en cada zona que la revisan en un segundo ([Authorize]).

Arquitectura Robusta

Integración nativa bajo estándares de Clean Architecture y SOLID.

Piénsalo así: *Cualquier guardia (instancia de la API) valida la pulsera: no hay un guardarropa central que se caiga.*

Alto Desempeño

Optimización en memoria, baja latencia y alta concurrencia.

Piénsalo así: *El guardia mira el holograma; no llama a la taquilla (base de datos) por cada persona.*

Seguridad Total

Protección perimetral de endpoints, datos sensibles y sesiones.

Piénsalo así: *Contraseñas licuadas (hash), pulseras selladas (firma) y zonas VIP (roles).*

Pilares técnicos fundamentales

New Horizons®
Centro de Capacitación

Mapa de la sesión: cinco pilares, cada uno con su imagen mental.

1		**Tokens JWT vs Sesiones de servidor**	*Guardarropa con ticket vs. pulsera de festival*
2		**Anatomía de un JWT: Header, Payload y Firma**	*Un pase de abordar con holograma*
3		**Password Hashing con PBKDF2 y Salt dinámico**	*La licuadora: no se puede “des-licuar”*
4		**Pipeline .NET 8: AddAuthentication y AddJwtBearer**	*El control de seguridad del aeropuerto*
5		**POST /api/auth/login y [Authorize]**	*La taquilla y la puerta VIP*

Implementación Técnica — Bloque 1

New Horizons
Centro de Capacitación

Sesion13.cs

```
// Implementación de referencia para la Sesión 13
// Autenticación moderna en APIs: Tokens JWT vs Sesiones de servidor
public class ConfiguracionSesion13 {
    public static void Configurar(IServiceCollection services) {
1       services.AddAuthentication(
            JwtBearerDefaults.AuthenticationScheme);
2       services.AddAuthorization();
    }
}
```

Las dos preguntas del guardia

Autenticación

¿Quién eres? Lee y valida la pulsera.

Autorización

¿Puedes entrar aquí? Revisa la zona.



El esqueleto del festival

Antes de contratar guardias que decidan quién entra, necesitas el lector que reconoce las pulseras.

1 AddAuthentication

Registra el esquema JWT Bearer: el lector de pulseras.

2 AddAuthorization

Registra las reglas de acceso que usará [Authorize].

Hoy completamos este esqueleto paso a paso: clave, token, hash, login y endpoints protegidos.

Práctica: Generación de Tokens JWT y Servicio

New Horizons
Centro de Capacitación

appsettings.json

```
{
  "Jwt": {
    "Key": "CLAVE-DEMO-NO-USAR-EN-PROD-0123456789abcdef",
    "Issuer": "https://api.demo-gastos.local",
    "Audience": "https://app.demo-gastos.local",
    "ExpiresMinutes": 60
  }
}
```

Paso a paso de la práctica

1	**appsettings**	Clave ≥ 256 bits, emisor y audiencia
2	**SymmetricSecurityKey**	Convierte la clave en bytes para firmar
3	**Claims del usuario**	Id, email y rol del empleado
4	**JwtSecurityTokenHandler**	Escribe el token como texto

El sello del notario

La Key es el sello con el que firmas cada pulsera. Si alguien lo copia, puede fabricar pulseras falsas.

1 Key

Mínimo 32 bytes (256 bits) para HMAC-SHA256.

2 Issuer

Quién emite la pulsera: tu API.

3 Audience

Para quién es válida: tu app Angular.

4 ExpiresMinutes

Cuánto dura la pulsera antes de caducar.

En producción, la Key va en variables de entorno o un gestor de secretos.

Sesiones de servidor vs. Tokens JWT

New Horizons®
Centro de Capacitación

	Sesión de servidor	Token JWT
	El guardarropa con ticket	*La pulsera del festival*
¿Dónde vive el estado?	En el servidor: te dan un ticket (cookie) y tu abrigo queda adentro.	En el cliente: la pulsera lleva tus datos y el sello que los protege.
Escalar a 3 servidores	Todos deben compartir el guardarropa (sticky sessions o caché).	Cualquier guardia valida el holograma sin preguntar a nadie.
Clientes ideales	Web tradicional en el mismo dominio.	SPA (Angular), apps móviles y microservicios.
Revocar antes de tiempo	Fácil: tiras el abrigo del guardarropa.	Difícil: la pulsera vale hasta que caduca → expiración corta.

Anatomía de un JWT

New Horizons®
Centro de Capacitación



Un JWT es un pase de abordar: tipo de pase, datos del pasajero y un holograma que prueba que nadie lo alteró. Tres partes separadas por puntos.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI0MiIsInJvbGUiOiJBcHJvYmFkb3IiLCJleHAiOjE3OTg3NjE2MDB9.q3Xv9mZ1kQ7…firma-demo…
```



Header

El tipo de pase

```
{ "alg": "HS256",
  "typ": "JWT" }
```

Algoritmo de firma y tipo de token.



Payload

Datos del pasajero

```
{ "sub": "42",
  "role": "Aprobador",
  "exp": 1798761600 }
```

Los claims: quién eres, tu rol, cuándo vence.



Signature

El holograma

```
HMACSHA256(
  header + "." + payload,
  clave_secreta)
```

Prueba que el pase salió de tu API y no fue alterado.

Claims estándar y personalizados

New Horizons®
Centro de Capacitación

Los claims son lo impreso en el pase: nombre, vuelo, asiento, grupo de abordaje. Unos los define el estándar; otros los agregas tú.

Estándar (registrados)

`iss`	Quién lo emitió	`api.demo-gastos.local`
`aud`	Para quién es	`app.demo-gastos.local`
`sub`	Id del usuario	`42`
`exp`	Cuándo vence	`1798761600`
`jti`	Id único del token	`b7e1…`

Personalizados

`role`	Rol para [Authorize]	`Aprobador`
`email`	Correo del usuario	`ana@demo.pe`
`centroCostoId`	Dato de negocio	`12`

El payload es una postal, no un sobre cerrado. Está en Base64, no cifrado: cualquiera lo lee. Nunca guardes contraseñas, DNI, sueldos ni tarjetas.

La firma: por qué nadie puede falsificarlo

New Horizons
Centro de Capacitación

La firma es el holograma del pase: se calcula con una clave que solo conoce tu API. Si alguien cambia una sola letra, el holograma deja de coincidir.



Token intacto

```
"role": "Aprobador"
```

La API recalcula la firma con su clave → coincide → 200 OK.

Token alterado

```
"role": "Admin"    ← editado a mano
```

La firma recalculada ya no coincide → 401 Unauthorized.

Por eso la API **no necesita consultar la base de datos** para confiar en el token: le basta su clave y la matemática.

Password hashing: por qué no texto plano

New Horizons
Centro de Capacitación



Hashear es licuar: puedes convertir una fresa en jugo, pero nunca reconstruir la fresa a partir del jugo. Para validar, licúas de nuevo y comparas.



Texto plano

Una sola filtración expone todas las cuentas (y las de otros sitios donde repiten clave).

Prohibido



MD5 / SHA-1

Son rápidos: una GPU prueba miles de millones por segundo y existen tablas precalculadas.

Obsoleto



PBKDF2 / BCrypt

Salt único por usuario + miles de iteraciones: lento a propósito para el atacante.

Recomendado

Misma contraseña, dos usuarios, dos hashes distintos (gracias al salt)

ana@demo.pe	`"Clave-Demo-2026"`	`+ salt: 8fQ2…`	→	`600000.8fQ2….Xa91kL…`
luis@demo.pe	`"Clave-Demo-2026"`	`+ salt: Zt7m…`	→	`600000.Zt7m….pQ03vB…`

Hashing con PBKDF2 y salt dinámico

New Horizons
Centro de Capacitación

PasswordHasher.cs

```
public static class PasswordHasher {
    private const int SaltSize = 16;      // 128 bits
    private const int HashSize = 32;      // 256 bits
    private const int Iterations = 600_000;

    public static string Hash(string password) {
1       byte[] salt = RandomNumberGenerator.GetBytes(SaltSize);
2       byte[] hash = Rfc2898DeriveBytes.Pbkdf2(
            password, salt, Iterations,
            HashAlgorithmName.SHA256, HashSize);

3       return $"{Iterations}." +
               $"{Convert.ToBase64String(salt)}." +
               $"{Convert.ToBase64String(hash)}";
    }
}
```



La receta secreta de la licuadora

Cada usuario recibe un ingrediente distinto (salt) y la licuadora da 600 000 vueltas (iteraciones).

1 Salt aleatorio por usuario

RandomNumberGenerator: criptográficamente seguro, nunca Random().

2 PBKDF2 + SHA-256

Rfc2898DeriveBytes.Pbkdf2 repite el hash 600 000 veces.

3 Se guarda la receta completa

iteraciones.salt.hash en una sola columna.

```
600000.8fQ2….Xa91kL…
```

Verificación segura de contraseñas

New Horizons
Centro de Capacitación

PasswordHasher.cs

```
public static bool Verify(string password, string stored) {
    var partes = stored.Split('.');
    int iterations = int.Parse(partes[0]);
    byte[] salt = Convert.FromBase64String(partes[1]);
    byte[] esperado = Convert.FromBase64String(partes[2]);

    byte[] calculado = Rfc2898DeriveBytes.Pbkdf2(
        password, salt, iterations,
        HashAlgorithmName.SHA256, esperado.Length);

    return CryptographicOperations.FixedTimeEquals(
        calculado, esperado);
}
```

1
2
3

No se “des-licua”

Tomas la misma receta (salt e iteraciones), licúas la clave que escribió el usuario y comparas los jugos.

1 Separar la receta

Lee iteraciones, salt y hash guardados.

2 Licuar de nuevo

Recalcula con los mismos parámetros.

3 Comparar en tiempo constante

FixedTimeEquals tarda lo mismo aunque falle en el primer byte.

Un portero que responde más rápido cuando fallas al inicio te está dando pistas: eso es un timing attack.

Pipeline .NET 8: AddAuthentication y AddJwtBearer

New Horizons®
Centro de Capacitación

Program.cs

```
var jwt = builder.Configuration.GetSection("Jwt");

builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o => {
        o.TokenValidationParameters = new() {
1           ValidateIssuer = true,
            ValidIssuer = jwt["Issuer"],
2           ValidateAudience = true,
            ValidAudience = jwt["Audience"],
3           ValidateLifetime = true,
4           ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(jwt["Key"]!)),
            ClockSkew = TimeSpan.FromMinutes(1)
        };
    });

builder.Services.AddAuthorization();
```

El control del aeropuerto

Antes de dejarte pasar revisa cuatro cosas de tu pase de abordar.

1. **Issuer**
   ¿Lo emitió nuestra aerolínea (API)?
2. **Audience**
   ¿Es para este vuelo (la app Angular)?
3. **Lifetime**
   ¿Sigue vigente? ClockSkew da 1 min de tolerancia.
4. **SigningKey**
   ¿El holograma es auténtico?

Orden del middleware de seguridad

New Horizons
Centro de Capacitación

Program.cs

```
var app = builder.Build();

app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Primero el DNI, luego la puerta

El guardia no puede decidir si entras a la zona VIP si todavía no sabe quién eres.



⚠ **Si inviertes el orden**, UseAuthorization ve a todos como anónimos y cada endpoint con [Authorize] responde 401, aunque el token sea válido.

TokenService: fabricando el JWT

New Horizons®
Centro de Capacitación

TokenService.cs

```
public string GenerarToken(Usuario u) {
    var claims = new List<Claim> {
        new(JwtRegisteredClaimNames.Sub, u.Id.ToString()),
        new(JwtRegisteredClaimNames.Email, u.Email),
        new(ClaimTypes.Role, u.Rol),
        new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
    };
    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
    var creds = new SigningCredentials(
        key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: _config["Jwt:Issuer"],
        audience: _config["Jwt:Audience"],
        claims: claims,
        expires: DateTime.UtcNow.AddMinutes(60),
        signingCredentials: creds);

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

La imprenta de pulseras

Imprime los datos, estampa el holograma, pone fecha de vencimiento y entrega la pulsera.

1. **Claims = datos impresos**

   Id, email, rol y un id único (jti).

2. **Key + creds = holograma**

   Firma HMAC-SHA256 con la clave de appsettings.

3. **expires = vencimiento**

   Usa siempre UtcNow, nunca la hora local.

4. **WriteToken = entregar**

   Devuelve el texto eyJhbGciOi…

Endpoint POST /api/auth/login

New Horizons
Centro de Capacitación

AuthController.cs

```
[ApiController]
[Route("api/auth")]
public class AuthController(AppDbContext db, ITokenService tokens)
    : ControllerBase {

    [HttpPost("login")]
1   [AllowAnonymous]
    public async Task<IActionResult> Login(LoginDto dto) {
        var u = await db.Usuarios
            .SingleOrDefaultAsync(x => x.Email == dto.Email);

2       if (u is null ||
            !PasswordHasher.Verify(dto.Password, u.PasswordHash))
            return Unauthorized();

3       return Ok(new { token = tokens.GenerarToken(u) });
    }
}
```

La taquilla del festival

Muestras tu entrada (credenciales) y te colocan la pulsera (token).

1. **[AllowAnonymous]**
   La taquilla está afuera: no pide pulsera.
2. **Mismo 401 para todo**
   No reveles si falló el email o la clave.
3. **Ok con el token**
   El cliente lo guarda y lo envía después.

```
POST /api/auth/login
{ "email": "ana@demo.pe", "password": "••••" }
← 200 OK
{ "token": "eyJhbGciOiJIUzI1NiIs…" }
```

Protección de controladores con [Authorize]

New Horizons®
Centro de Capacitación

SolicitudesController.cs

```
[ApiController]
[Route("api/solicitudes")]
1 [Authorize]
public class SolicitudesController : ControllerBase {

    [HttpGet]
    public IActionResult Listar() { /* ... */ }

    [HttpPost("{id}/aprobar")]
2   [Authorize(Roles = "Aprobador")]
    public IActionResult Aprobar(int id) { /* ... */ }

    [HttpGet("estado")]
3   [AllowAnonymous]
    public IActionResult Estado() => Ok("API operativa");
}
```



Zonas del festival

General (cualquier pulsera), VIP (pulsera dorada) y entrada libre.

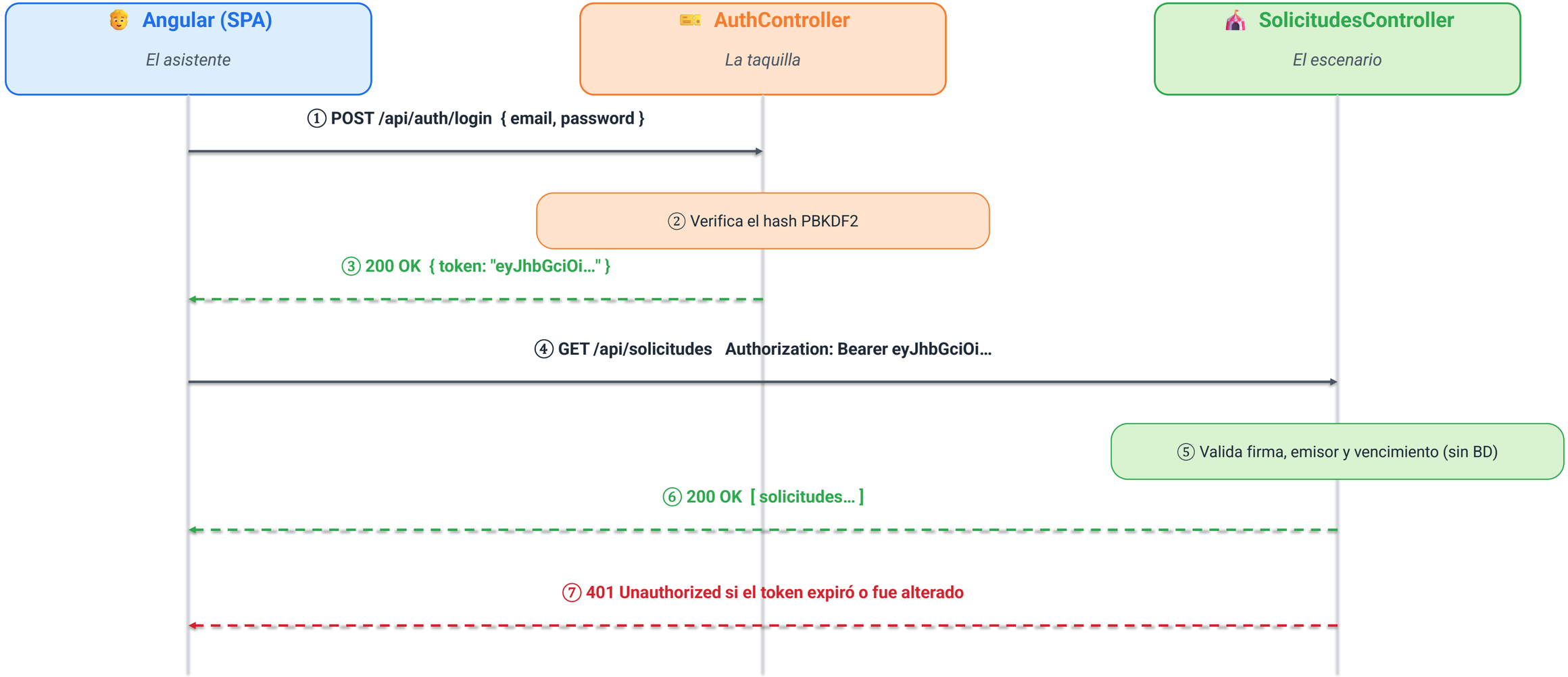
¿Qué responde la API?

	Listar	Aprobar	Estado
Sin token	401	401	200
Empleado	200	403	200
Aprobador	200	200	200

401 No sé quién eres (falta o falla el token).

403 Sé quién eres, pero no tienes el rol.

Flujo completo de autenticación



Errores comunes y cómo evitarlos

New Horizons®
Centro de Capacitación

✗ Evita esto	✓ Haz esto
Omitir la validación estricta de parámetros en el servidor	Validar siempre los contratos DTO en cada llamada HTTP
Almacenar información sensible en repositorios públicos	Utilizar variables de entorno seguras en desarrollo y producción
Desacoplar la UI de las políticas de seguridad del backend	Mantener arquitectura standalone y tipado estricto
Guardar contraseñas en texto plano o con MD5/SHA-1	PBKDF2 o BCrypt con salt e iteraciones
Poner datos sensibles en el payload del JWT	Solo identificadores, roles y lo mínimo necesario

Buenas prácticas de seguridad con JWT

New Horizons
Centro de Capacitación



Un JWT es dinero en efectivo: quien lo tiene, lo gasta. Nadie pregunta si es tuyo. Cúidalo como tal.



Vida corta

Expiración de 15 a 60 minutos y refresh token para renovarla.

Si te lo roban, dura poco.



Solo por HTTPS

Nunca envíes el token por HTTP ni en la URL (query string).

El efectivo no se manda en sobre abierto.



Secretos fuera del código

Clave en variables de entorno o gestor de secretos; rótala periódicamente.

El sello del notario no se deja en la mesa.



Payload mínimo

Solo id, rol y lo indispensable. Nada que no quieras ver publicado.

La postal la lee el cartero.

Resumen de la Sesión 13

New Horizons®
Centro de Capacitación

Autenticación moderna

JWT proporciona autenticación stateless escalable ideal para arquitecturas SPA y móviles.

≈ La pulsera del festival

Anatomía de un JWT

Los claims transmiten identidad y roles sin saturar la base de datos en cada petición.

≈ El pase de abordar

Password Hashing

El hashing con PBKDF2 o BCrypt protege las credenciales ante eventuales filtraciones.

≈ La licuadora

Pipeline .NET 8

El middleware de autenticación valida firma y expiración de forma ultrarrápida en memoria.

≈ El control del aeropuerto

Excelencia

El atributo [Authorize] blinda los endpoints del backend contra accesos no autorizados.

≈ La puerta VIP

Full Stack

Solución enterprise lista para producción.

≈ El festival completo

PREGUNTAS...

New Horizons®
Centro de Capacitación

1. ¿Por qué preferimos JWT sobre sesiones con cookies en arquitecturas desacopladas?
2. ¿Qué información es seguro almacenar dentro del payload de un token JWT?
3. ¿Por qué nunca debemos guardar contraseñas en texto plano ni usar MD5/SHA1?
4. ¿Cómo valida ASP.NET Core la autenticidad del token sin consultar la base de datos?
5. ¿Qué sucede cuando un token JWT expira y cómo manejamos la caducidad?

New Horizons
Centro de Capacitación

Conclusiones



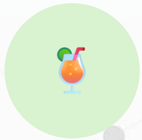
Sin estado, sin límites

JWT proporciona autenticación stateless escalable ideal para arquitecturas SPA y móviles.



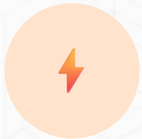
Identidad que viaja

Los claims transmiten identidad y roles sin saturar la base de datos en cada petición.



Contraseñas licuadas

El hashing con PBKDF2 o BCrypt protege las credenciales ante eventuales filtraciones.



Validación en memoria

El middleware de autenticación valida firma y expiración de forma ultrarrápida en memoria.



Endpoints blindados

El atributo [Authorize] blinda los endpoints del backend contra accesos no autorizados.

GRACIAS